

Algoritmos e Estrutura de Dados

Projeto Integrador · Unidade 1

HemoTrack / Rota Vital

Documentação técnica da implementação em C++ das estruturas de domínio: estoque de bolsas, requisições hospitalares e histórico de operações, com alocação manual de memória.

*struct Bolsa**

lista encadeada · fila · pilha · recursão





Sumário

Guia de leitura do documento

01	Apresentação e objetivo	O que esta entrega demonstra	3
02	Duas disciplinas, duas versões	Por que o C++ é diferente do Spring Boot	4
03	Reformulação das histórias de usuário	US01a US07 no escopo de AED	6
04	Arquitetura do projeto	Arquivos, dependências e compilação separada	9
05	Decisões técnicas	malloc/free, char[], datas e leitura segura	11
06	Modelagem dos nós	As structs de tipos.h	14
07	Lista encadeada: o estoque	estoque.h / estoque.cpp	17
08	Fila: as requisições	fila.h / fila.cpp	21
09	Pilha: o histórico e o desfazer	historico.h / historico.cpp	24
10	Recursão aplicada	Busca e contagens recursivas	27
11	Casos de uso em detalhe	servicos.cpp	30
12	Gerenciamento de memória	Quem aloca, quem libera	34
13	Complexidade das operações	Análise assintótica	36
14	Execução e demonstração	OnlineGDB, g++ e roteiro de teste	37
15	Conclusão e evoluções	Síntese e próximos passos	39

Identificação da entrega

Disciplina	Algoritmos e Estrutura de Dados
Entrega	Projeto Integrador, Unidade 1: estruturas básicas de domínio
Sistema	HemoTrack / Rota Vital: gestão de estoque de hemocomponentes e requisições hospitalares
Linguagem	C++17, com alocação manual (<code>malloc</code> / <code>free</code>) e ponteiros explícitos
Código-fonte	12 arquivos: <code>main.cpp</code> , <code>tipos.h</code> , <code>utils</code> , <code>estoque</code> , <code>fila</code> , <code>historico</code> , <code>servicos</code>

01 Apresentação e objetivo

O que esta entrega demonstra

O **HemoTrack** (também chamado de **Rota Vital**) é um sistema de apoio à cadeia de suprimento de sangue: o hemocentro mantém um estoque de bolsas de hemocomponentes, os hospitais enviam requisições e os operadores separam o material para despacho. Este documento descreve a **versão em C++** construída para a disciplina de **Algoritmos e Estrutura de Dados**, cujo foco não é a regra de negócio em si, mas a forma como os dados são organizados na memória.

O enunciado desta entrega pede a implementação das estruturas básicas de domínio do sistema com três exigências centrais: o **estoque** deve ser uma **lista**, as **requisições** devem formar uma **fila** e, se aplicável, o **histórico de operações** deve ser uma **pilha**, tudo com alocação manual de memória (`malloc` / `free`) e manipulação explícita de ponteiros. O enunciado também proíbe expressamente lógica de roteirização, FEFO, hash e compatibilidade ABO/Rh.

Como cada requisito foi atendido

Requisito do professor	Como foi atendido no código	Onde
Estoque como lista	Lista simplesmente encadeada de nós <code>Bolsa</code> , com ponteiros para <code>inicio</code> e <code>fim</code> .	<code>estoque.cpp</code>
Requisições como fila	Fila FIFO de nós <code>Requisicao</code> : entrada sempre no fim (<code>enqueue</code>), saída sempre no início (<code>dequeue</code>).	<code>fila.cpp</code>
Histórico como pilha	Pilha LIFO de nós <code>Operacao</code> ; cada ação que altera o sistema é empilhada e pode ser desfeita com <code>pop</code> .	<code>historico.cpp</code>
Alocação manual	Todo nó nasce com <code>malloc(sizeof(...))</code> , tem o retorno testado contra <code>NULL</code> e morre com <code>free</code> .	todos
Ponteiros explícitos	Encadeamento por campos <code>prox</code> , acesso com <code>-></code> e estruturas passadas por ponteiro para serem alteradas no lugar.	todos
Recursão (conteúdo da unidade)	Busca por código e contagens implementadas recursivamente, com caso base e passo recursivo explícitos.	<code>estoque.cpp</code> , <code>historico.cpp</code>
Sem FEFO, hash, ABO/Rh, rotas	Nenhuma dessas lógicas existe no código. O tipo sanguíneo é guardado apenas como dado informativo.	(ver seção 02)

3

estruturas de dados implementadas do zero

4

funções recursivas com caso base explícito

7

histórias de usuário cobertas pelo menu

0

vazamentos ou acessos inválidos nos testes com AddressSanitizer

02 Duas disciplinas, duas versões

Por que o C++ é diferente do Spring Boot

O HemoTrack é um projeto que atravessa mais de uma disciplina. Na disciplina de **Programação Orientada a Objetos**, o sistema foi desenvolvido em **Java com Spring Boot**, e ali a equipe teve liberdade para usar recursos de alto nível. Na disciplina de **Algoritmos e Estrutura de Dados**, o mesmo domínio foi reimplementado em **C++** sob regras bem diferentes. Esta seção explica essa diferença, porque ela justifica todas as escolhas feitas no restante do documento.

O que era permitido na versão Java / Spring Boot

Em POO, o objetivo era modelar o sistema com classes, camadas e frameworks. Nesse contexto, a equipe teve permissão para usar, se quisesse, recursos mais avançados que as histórias de usuário originais previam:

- **FEFO** (*First Expired, First Out*): priorizar automaticamente as bolsas com vencimento mais próximo;
- **Compatibilidade ABO/Rh**: decidir automaticamente quais tipos sanguíneos de doador servem para cada receptor;
- **Hash** (por exemplo `HashMap`) para buscas em tempo constante médio;
- Coleções prontas da linguagem, persistência em banco, API REST e recursos como roteirização por grafos (Dijkstra), telemetria e análise preditiva, conforme as histórias originais.

Por que isso não se aplica em AED

Em AED, o professor avalia se o aluno **sabe construir e manipular as estruturas por dentro**: criar um nó, ligar ponteiros, tratar lista vazia, não perder a referência ao remover, liberar memória. Recursos como hash e coleções prontas escondem exatamente isso. Regras como FEFO e ABO/Rh, por sua vez, acrescentam complexidade de negócio sem exercitar o conteúdo da unidade. Além disso, a entrega exige uma versão Java equivalente com justificativa de equivalência linha a linha. Isso só é possível se a lógica for a mesma nas duas linguagens, e não há equivalente direto de um projeto Spring Boot em C puro.

Por isso o enunciado proíbe explicitamente essas lógicas. **Como no caso desta disciplina elas não eram permitidas, aplicamos o domínio de forma diferente**: mantivemos os conceitos do HemoTrack (bolsas, requisições, status, validade, hospitais) e redesenhamos as funcionalidades para que cada uma exercite uma estrutura estudada.

Aspecto	Java / Spring Boot (POO): permitido	C++ (AED): o que foi feito
Foco da disciplina	Modelagem orientada a objetos, camadas, frameworks	Organização dos dados na memória e algoritmos sobre as estruturas
Estoque	Coleções da linguagem e persistência em banco	Lista encadeada construída à mão, nó a nó
Ordem de atendimento	Podia priorizar por urgência ou validade	Fila FIFO pura: quem chega primeiro é atendido primeiro

Aspecto	Java / Spring Boot (P00): permitido	C++ (AED): o que foi feito
Escolha da bolsa	Automática, por FEFO e compatibilidade ABO/Rh	Manual, pelo operador; o sistema só confere se o hemocomponente é o mesmo pedido
Tipo sanguíneo	Participava da regra de compatibilidade	Dado informativo; só o formato é validado (A+, O-, ...)
Busca por código	Hash, O(1) médio	Percurso recursivo na lista, O(n)
Histórico	Registros em banco / logs	Pilha com operação de desfazer
Rota de entrega	Dijkstra sobre grafo viário	Fora do escopo (grafos não fazem parte da unidade)
Memória	Coletor de lixo (garbage collector)	<code>malloc</code> e <code>free</code> explícitos, com dono definido para cada nó

Não é uma versão “menor”, é um recorte deliberado

A versão C++ não é uma simplificação por falta de capacidade técnica: é a mesma ideia de sistema reorganizada para responder à pergunta que esta disciplina faz, que é **“você sabe implementar e justificar as estruturas?”**. As funcionalidades avançadas continuam existindo na versão Spring Boot, onde eram permitidas. A versão Java que acompanha esta entrega para o documento de equivalência deve seguir a lógica deste código C++, e não a do Spring Boot.

03 Reformulação das histórias de usuário

US01 a US07 no escopo de AED

As sete histórias de usuário da primeira entrega foram revisadas. As que já envolviam apenas cadastro e registro foram mantidas, retirando qualquer regra proibida. As que dependiam de FEFO, ABO/Rh, grafos, telemetria ou estatística foram substituídas por funcionalidades do mesmo domínio que exercitam lista, fila, pilha ou recursão. As duas últimas foram adaptadas: mantêm a intenção gerencial original (acompanhar o que acontece no hemocentro), mas com recursos compatíveis com a disciplina.

História original	Situação	História na versão C++	Estrutura
US01 Requisição emergencial	MANTIDA	Registrar requisição hospitalar, sem regra de compatibilidade	Fila
US02 Cadastro de bolsas	MANTIDA	Cadastrar bolsa no estoque, com validação de validade	Lista
US03 Alocação FEFO + ABO/Rh	SUBSTITUÍDA	Atender requisições por ordem de chegada, com seleção manual das bolsas	Fila + Lista
US04 Rota por Dijkstra	SUBSTITUÍDA	Consultar estoque e buscar bolsa por código	Lista + recursão
US05 Telemetria de temperatura	SUBSTITUÍDA	Descartar bolsa do estoque, com motivo	Lista (remoção)
US06 Painel gerencial	ADAPTADA	Histórico de operações com desfazer	Pilha
US07 Demanda preditiva	ADAPTADA	Relatório consolidado com contagens (sem estatística)	Recursão sobre lista e pilha

Histórias reescritas no padrão BDD

Os cenários abaixo seguem o formato Dado-Quando-Então da primeira entrega e correspondem ao comportamento real do programa. Ao final de cada cartão está indicado onde a história está implementada.

US01 Registrar requisição hospitalar

Como profissional de saúde hospitalar, quero registrar uma requisição de hemocomponente para que ela entre na fila de atendimento do hemocentro.

CENÁRIO 1: REQUISIÇÃO VÁLIDA

- **Dado** que o profissional está no menu de requisições
- **Quando** informa hospital, hemocomponente, tipo sanguíneo, quantidade (1 a 10) e urgência
- **Então** a requisição entra no **fim da fila** com status "Pendente de Alocação"
- **E** o sistema mostra o identificador **REQ-0001** e a posição na fila

CENÁRIO 2: CAMPO OBRIGATÓRIO AUSENTE

- **Quando** o tipo sanguíneo (ou o hospital) é deixado em branco
- **Então** a requisição **não** é registrada e o sistema informa qual campo é obrigatório

Função `registrarRequisicao()` · estrutura: **fila** (`enqueue`) · empilha `OP_NOVA_REQUISICAO`

US02 Cadastrar bolsa no estoque

Como operador do hemocentro, quero registrar a entrada de bolsas informando código, hemocomponente, tipo ABO/Rh e datas, para manter o estoque atualizado e rastreável.

CENÁRIO 1: BOLSA VÁLIDA

- **Quando** insere `BOL-9821`, “Plaquetas”, “A+”, coleta válida e validade futura
- **Então** a bolsa é inserida no **fim da lista** com status “Disponível”
- **E** o saldo disponível do hemocomponente é exibido já atualizado

CENÁRIO 2: BOLSA VENCIDA OU CÓDIGO REPETIDO

- **Quando** a validade é anterior à data atual, ou o código já existe
- **Então** o cadastro é bloqueado (“Data de validade inválida. Não é permitido cadastrar bolsas vencidas.”)

Função `cadastrarBolsa()` · estrutura: **lista** (`inserirFimEstoque`) · empilha `OP_CADASTRO_BOLSA`

US03 Atender requisições por ordem de chegada

Como operador do hemocentro, quero atender a requisição mais antiga da fila escolhendo as bolsas a separar, para que nenhum pedido seja passado para trás.

CENÁRIO: ATENDIMENTO COM ESTOQUE SUFICIENTE

- **Dado** que `REQ-0001` está no início da fila pedindo 2 bolsas de Plaquetas
- **Quando** o operador informa os códigos de 2 bolsas disponíveis de Plaquetas
- **Então** as bolsas passam a “Reservada para Despacho” e a requisição sai da fila (`dequeue`)
- **E** se o estoque for insuficiente, a requisição permanece na fila sem alterações

Função `atenderProximaRequisicao()` · estruturas: **fila + lista** · empilha `OP_ATENDIMENTO`

US04 Consultar estoque e buscar bolsa

Como operador, quero listar o estoque e localizar uma bolsa pelo código para verificar sua situação.

CENÁRIO: BUSCA POR CÓDIGO

- **Quando** o operador informa um código existente
- **Então** o sistema exibe os dados completos da bolsa; caso contrário, informa que não a encontrou

Funções `listarEstoque()` e `consultarBolsa()` · **lista + busca recursiva**

US05 Descartar bolsa do estoque

Como operador, quero remover uma bolsa inutilizada informando o motivo, para que o estoque reflita apenas material aproveitável.

CENÁRIO: DESCARTE PERMITIDO

- **Quando** o operador informa uma bolsa "Disponível" e escolhe o motivo
- **Então** o nó é removido da lista e liberado com `free`, e uma cópia vai para o histórico
- **E** bolsas "Reservadas para Despacho" não podem ser descartadas

Função `descartarBolsa()` · lista(`removerBolsa`) · empilha `OP_DESCARTE_BOLSA`

US06 Histórico de operações e desfazer

Como gestor, quero ver as operações realizadas e desfazer a última, para corrigir erros de digitação sem retrabalho.

CENÁRIO: DESFAZER

- **Dado** que a última operação foi um atendimento
- **Quando** o gestor escolhe "Desfazer última operação"
- **Então** as bolsas voltam a "Disponível" e a requisição volta para o início da fila

Funções `exibirHistorico()` e `desfazerUltimaOperacao()` · pilha(`push` / `pop`)

US07 Relatório consolidado

Como gestor, quero um resumo do estoque e das operações para identificar gargalos.

CENÁRIO: GERAÇÃO DO RELATÓRIO

- **Quando** o gestor escolhe "Relatório consolidado"
- **Então** vê totais por status, disponíveis por hemocomponente, requisições pendentes, atendidas e descartes

Função `gerarRelatorio()` · **contagens recursivas** sobre a lista e sobre a pilha

04 Arquitetura do projeto

Arquivos, dependências e compilação separada

O código foi dividido em **módulos com cabeçalho (.h)** e **implementação (.cpp)**. O cabeçalho diz *o que* um módulo oferece; a implementação diz *como*. Essa separação espelha o conceito de **Tipo Abstrato de Dados** visto em aula: quem usa a fila, por exemplo, conhece apenas `enqueue`, `dequeue` e `peekFila`, sem depender de como os ponteiros são manipulados por dentro.

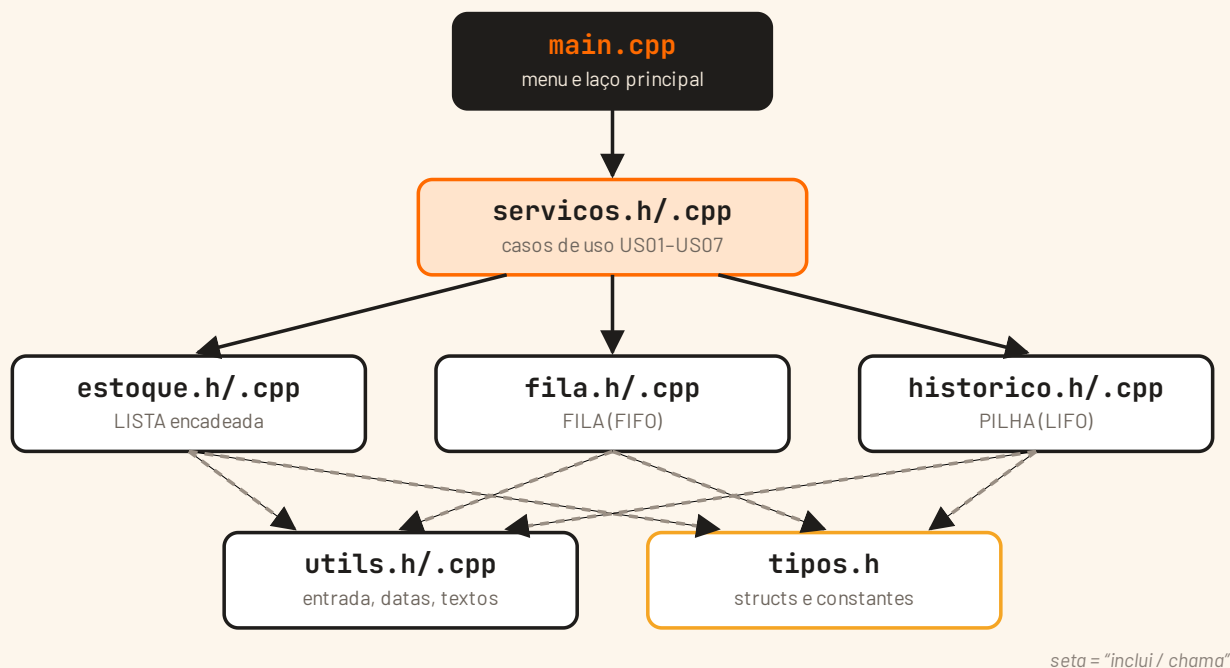


Figura 1. Dependências entre os módulos. Os casos de uso são a única camada que conhece as três estruturas ao mesmo tempo.

Arquivo	Responsabilidade	Principais funções
tipos.h	Constantes de tamanho e de status, declaração das listas de opções e definição das <i>structs</i> (nós e descritores das três estruturas).	(apenas tipos)
utils.h / .cpp	Funções de apoio sem relação com estruturas: leitura segura do teclado, validação e formatação de datas, cópia segura de texto.	<code>lerLinha</code> , <code>lerInteiro</code> , <code>lerData</code> , <code>copiar</code>
estoque.h / .cpp	Lista encadeada de bolsas: criação, inserção, remoção, busca, contagens e liberação.	<code>inserirFimEstoque</code> , <code>removerBolsa</code> , <code>buscarBolsaRec</code>
fila.h / .cpp	Fila de requisições: enfileirar, desenfileirar, consultar a frente e liberação.	<code>enqueue</code> , <code>dequeue</code> , <code>peekFila</code>
historico.h / .cpp	Pilha de operações: empilhar, desempilhar, contar e liberação.	<code>push</code> , <code>pop</code> , <code>contarOperacoesRec</code>

Arquivo	Responsabilidade	Principais funções
<code>servicos.h / .cpp</code>	Casos de uso: combinam as três estruturas para realizar cada história, com validações e mensagens.	<code>cadastrarBolsa</code> , <code>atenderProximaRequisicao</code> , ...
<code>main.cpp</code>	Cria as três estruturas, exibe o menu, despacha a opção escolhida e libera toda a memória ao sair.	<code>main</code> , <code>exibirMenu</code>

Detalhes de compilação separada

Guardas de inclusão. Cada cabeçalho é protegido por `#ifndef` / `#define` / `#endif`. Como `tipos.h` é incluído por quase todos os arquivos, sem a guarda as `structs` seriam definidas mais de uma vez na mesma unidade de compilação e o compilador acusaria redefinição.

Variáveis globais com `extern`. As listas de opções (hemocomponentes, tipos, urgências, motivos) e a flag `fimEntrada` são usadas em vários arquivos. Em `tipos.h` elas são apenas *declaradas* com `extern`; a *definição* (onde a memória existe de fato) acontece uma única vez, em `utils.cpp`. Se fossem definidas no cabeçalho, cada `.cpp` teria sua própria cópia e o *linker* acusaria *multiple definition*, o mesmo tipo de erro que acontece quando dois arquivos definem `main()`.

`tipos.h`

linhas 25–35

```
// Listas de opções (definidas em utils.cpp)
extern const char* HEMOCOMPONENTES[];
extern const int QTD_HEMOCOMPONENTES;
extern const char* TIPOS_SANGUINEOS[];
extern const int QTD_TIPOS;
extern const char* URGENCIAS[];
extern const int QTD_URGENCIAS;
extern const char* MOTIVOS_DESCARTE[];
extern const int QTD_MOTIVOS;

extern bool fimEntrada; // true quando a entrada padrão termina (EOF)
```

Uma única função `main`. Apenas `main.cpp` contém `main()`. Ambientes como o OnlineGDB compilam todos os `.cpp` abertos juntos, por isso não pode haver outro arquivo com `main` no projeto.

05 Decisões técnicas

malloc/free, char[], datas e leitura segura

Antes de entrar nas estruturas, vale explicar quatro decisões que aparecem em todo o código. Elas não são detalhes de estilo: cada uma resolve um problema concreto que surge quando se usa alocação manual em C++.

5.1 · malloc/free em vez de new/delete

O enunciado exige `malloc` e `free`, que são as funções de alocação dinâmica da linguagem C (também disponíveis em C++ via `<cstdlib>`). `malloc(sizeof(Bolsa))` reserva exatamente os bytes de um nó no *heap* e devolve um ponteiro genérico (`void*`), que é convertido para `Bolsa*`. Se não houver memória, devolve `NULL`, e **todo** `malloc` do projeto testa esse retorno antes de usar o ponteiro. Cada bloco alocado é devolvido com `free` exatamente uma vez.

5.2 · Textos como char[], e não std::string

Esta é a decisão mais importante do projeto. `malloc` apenas reserva bytes: ele **não executa construtores**. Uma `std::string` é um objeto que precisa ser construído (ela guarda internamente ponteiro, tamanho e capacidade). Se uma `std::string` estivesse dentro de um nó criado por `malloc`, ela conteria lixo de memória, e a primeira atribuição poderia travar o programa. O mesmo vale para `free`, que não executa destrutores. Por isso todos os campos de texto são vetores de tamanho fixo (`char codigo[TAM_COD]`), que não precisam de construção e são copiados junto com a *struct*.

CONSEQUÊNCIA PRÁTICA

Como os nós contêm apenas tipos simples (`int`, `char[]`, ponteiros), uma atribuição como `op->bolsa = *nova;` copia a *struct* inteira byte a byte, de forma segura. Isso é usado pela pilha para guardar cópias.

5.3 · Cópia segura de texto

`strncpy` sozinho é perigoso: se a origem for maior que o destino, ele não coloca o terminador `'\0'`, e a string resultante “vaza” pela memória vizinha. A função `copiar` limita a cópia a `tamanho - 1` e força o terminador na última posição. Assim, um nome de hospital longo é apenas truncado, nunca estoura o vetor.

utils.cpp

linhas 30-33

```
void copiar(char* destino, const char* origem, int tamanho) {
    strncpy(destino, origem, tamanho - 1);
    destino[tamanho - 1] = '\0';
}
```

5.4 · Datas representadas como inteiro aaaammdd

Uma data como 31/12/2030 é guardada como o inteiro `20301231`. Nesse formato, a ordem numérica coincide com a ordem cronológica, então comparar datas vira uma simples comparação de inteiros (`validade < hoje`). A data atual vem de `time()` e `localtime()`, e `dataExiste` rejeita datas impossíveis como 31/02, considerando anos bissextos.

utils.cpp

linhas 77-77

```
int dataParaInt(int dia, int mes, int ano) { return ano * 10000 + mes * 100 + dia; }
```

utils.cpp

linhas 85-91

```
bool dataExiste(int dia, int mes, int ano) {  
    if (ano < 1900 || mes < 1 || mes > 12 || dia < 1) return false;  
    int dias[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};  
    bool bissexto = (ano % 4 == 0 && ano % 100 != 0) || ano % 400 == 0;  
    if (bissexto) dias[1] = 29;  
    return dia <= dias[mes - 1];  
}
```

ISTO NÃO É FEFO

A comparação de datas é usada apenas para **validar** o cadastro (impedir bolsa vencida ou com coleta no futuro). Em nenhum momento o sistema ordena ou escolhe bolsas pela validade, que seria a regra FEFO proibida.

5.5 · Leitura robusta do teclado

Misturar `cin >> numero` com leitura de linhas costuma deixar um `'\n'` pendente no buffer e “pular” a próxima pergunta. Para evitar isso, **toda** entrada é lida como linha inteira por `lerLinha`, e números são convertidos depois com `strtol`. `lerInteiro` repete a pergunta até receber um número válido dentro do intervalo, e só aceita a conversão se a linha inteira for numérica (o ponteiro `resto` precisa chegar ao fim da string).

utils.cpp

linhas 63-75

```
int lerInteiro(const char* prompt, int minimo, int maximo) {  
    char buffer[32];  
    while (true) {  
        lerLinha(prompt, buffer, sizeof(buffer));  
        if (fimEntrada) return INT_MIN;  
        char* resto;  
        long valor = strtol(buffer, &resto, 10);  
        if (resto != buffer && *resto == '\0' && valor >= minimo && valor <= maximo)  
            return (int)valor;  
        cout << " Valor inválido. Digite um número entre " << minimo  
            << " e " << maximo << ".\n";  
    }  
}
```

Se a entrada terminar (fim de arquivo, útil em testes automatizados), `LerLinha` liga a flag global `fimEntrada`. Cada caso de uso verifica essa flag após as leituras e retorna sem alterar nada, e o laço do `main` encerra liberando toda a memória.

5.6 · Estruturas passadas por ponteiro

As três estruturas são variáveis locais de `main`, e as funções as recebem por ponteiro (`ListaEstoque* estoque`). Assim, `inserirFimEstoque(&estoque, nova)` altera a lista original, e não uma cópia. É a mesma ideia do exemplo `swap(int* a, int* b)` da aula de ponteiros, aplicada a estruturas maiores.

06 Modelagem dos nós

As structs de tipos.h

Todas as estruturas do projeto seguem o mesmo padrão da aula de listas: um **nó** que guarda os dados mais um ponteiro para o próximo nó, e um **descriptor** que guarda os ponteiros de acesso à estrutura (início, fim ou topo) e o tamanho. Os tipos ficam centralizados em `tipos.h`.

6.1 - O nó Bolsa e o descriptor ListaEstoque

tipos.h

linhas 37-52

```
/* ----- LISTA: estoque de bolsas ----- */
typedef struct Bolsa {
    char codigo[TAM_COD];
    char hemocomponente[TAM_TXT];
    char tipoSanguineo[TAM_TIPO]; // apenas informativo
    int dataColeta;                // aaaammdd
    int dataValidade;              // aaaammdd
    char status[TAM_TXT];
    struct Bolsa* prox;
} Bolsa;

typedef struct ListaEstoque {
    Bolsa* inicio;
    Bolsa* fim;
    int tamanho;
} ListaEstoque;
```

`sizeof(Bolsa) = 136 bytes (64 bits): 128 bytes de dados + 8 do ponteiro`



dados da bolsa: copiados com `copiar()` e atribuições simples

próxima Bolsa (ou NULL)

Figura 2. Como um nó **Bolsa** fica organizado na memória. O único campo que “aponta para fora” é **prox**.

O descriptor **ListaEstoque** guarda três informações. **inicio** é a porta de entrada da lista: perdê-lo significa perder todos os nós. **fim** aponta para o último nó e existe para que a inserção no final seja imediata, sem percorrer a lista. **tamanho** evita contar os nós a cada consulta.

6.2 - 0 nó Requisicao e o descritor FilaRequisicoes

tipos.h

linhas 54-73

```
/* ----- FILA: requisições ----- */
typedef struct Requisicao {
    int id;
    char hospital[TAM_TXT];
    char hemocomponente[TAM_TXT];
    char tipoSanguineo[TAM_TIPO];
    int quantidade;
    char urgencia[TAM_TXT];
    char status[TAM_TXT];
    char bolsasAlocadas[MAX_BOLSAS_REQ][TAM_COD];
    int qtdAlocada;
    struct Requisicao* prox;
} Requisicao;

typedef struct FilaRequisicoes {
    Requisicao* inicio; // dequeue
    Requisicao* fim;    // enqueue
    int tamanho;
    int proximoId;
} FilaRequisicoes;
```

A fila precisa de **dois ponteiros**, exatamente como na tabela “Pilha × Fila” da aula: **inicio** (de onde as requisições saem) e **fim** (onde entram). O campo **proximoId** gera identificadores sequenciais e nunca é decrementado, para que um identificador nunca seja reaproveitado. O vetor **bolsasAlocadas** registra quais bolsas foram separadas no atendimento, informação necessária para desfazê-lo.

6.3 - 0 nó Operacao e o descritor PilhaHistorico

tipos.h

linhas 75-94

```
/* ----- PILHA: histórico ----- */
typedef enum {
    OP_CADASTRO_BOLSA,
    OP_DESCARTE_BOLSA,
    OP_NOVA_REQUISICAO,
    OP_ATENDIMENTO
} TipoOperacao;

typedef struct Operacao {
    TipoOperacao tipo;
    char descricao[TAM_DESC];
    Bolsa bolsa; // cópia (cadastro/descarte)
    Requisicao requisicao; // cópia (registro/atendimento)
    struct Operacao* prox;
} Operacao;

typedef struct PilhaHistorico {
    Operacao* topo;
    int tamanho;
} PilhaHistorico;
```

A pilha só precisa do `topo`, pois inserção e remoção acontecem sempre na mesma ponta. O detalhe importante está no nó: `Operacao` guarda uma **cópia por valor** da bolsa e da requisição envolvidas, e não um ponteiro para elas.

Por que cópia, e não ponteiro?

Quando uma bolsa é descartada ou uma requisição é atendida, o nó original é liberado com `free`. Se o histórico guardasse um ponteiro para esse nó, ele passaria a apontar para memória já devolvida (**ponteiro pendente**, ou *dangling pointer*), e ler esse endereço é comportamento indefinido. É o cuidado “nunca acessar memória após liberar” da aula de ponteiros. Com a cópia dentro do nó da pilha, os dados continuam válidos pelo tempo que a operação existir no histórico. O custo é o tamanho do nó (664 bytes em 64 bits), aceitável para o volume de uma sessão.

O `enum TipoOperacao` identifica o que precisa ser revertido em cada caso. Ele permite que uma única pilha armazene operações de naturezas diferentes e que o `desfazer` decida, com um `switch`, qual reversão aplicar.

07 Lista encadeada: o estoque

estoque.h / estoque.cpp

O estoque é uma **lista simplesmente encadeada**: cada bolsa conhece apenas a próxima. A lista foi escolhida porque o número de bolsas muda o tempo todo (entram por cadastro, saem por descarte) e, como visto em aula, a lista não exige tamanho máximo definido na criação e não precisa deslocar elementos ao inserir ou remover.

7.1 - Criação de um nó

estoque.cpp

Linhas 16-31

```
Bolsa* criarBolsa(const char* codigo, const char* hemocomponente, const char* tipo,
                  int dataColeta, int dataValidade, const char* status) {
    Bolsa* nova = (Bolsa*)malloc(sizeof(Bolsa));
    if (nova == NULL) {
        cout << " Erro: memória insuficiente para criar a bolsa.\n";
        return NULL;
    }
    copiar(nova->codigo, codigo, TAM_COD);
    copiar(nova->hemocomponente, hemocomponente, TAM_TXT);
    copiar(nova->tipoSanguineo, tipo, TAM_TIPO);
    nova->dataColeta = dataColeta;
    nova->dataValidade = dataValidade;
    copiar(nova->status, status, TAM_TXT);
    nova->prox = NULL;
    return nova;
}
```

- 1 `malloc(sizeof(Bolsa))` reserva os 136 bytes do nó no *heap*; o resultado é convertido para `Bolsa*`.
- 2 O retorno é comparado com `NULL`. Se a memória acabou, a função avisa e devolve `NULL`, e quem chamou desiste da operação sem alterar a lista.
- 3 Os textos são copiados com `copiar` (seção 5.3) e os inteiros por atribuição.
- 4 `nova->prox = NULL`: o nó nasce “solto”. Deixar `prox` sem inicializar seria acessar um ponteiro não inicializado, um dos cuidados listados na aula de ponteiros.

Criar e inserir são funções separadas de propósito. O desfazer de um descarte, por exemplo, recria a bolsa a partir da cópia guardada na pilha e reaproveita a mesma inserção.

7.2 · Inserção no fim

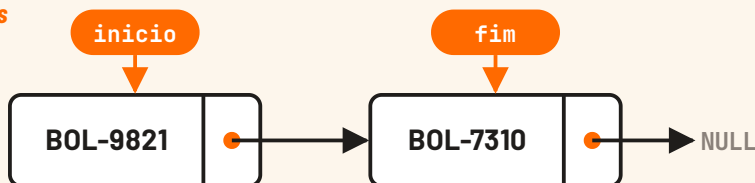
estoque.cpp

linhas 33-43

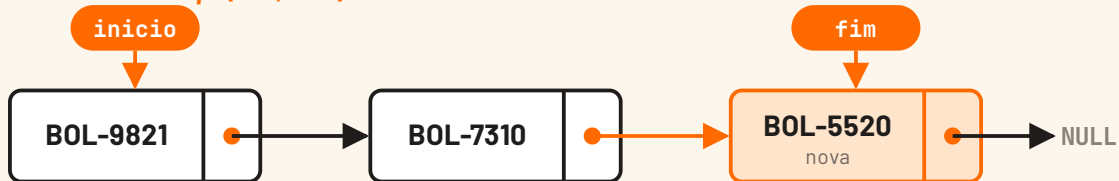
```
void inserirFimEstoque(ListaEstoque* lista, Bolsa* nova) {
    nova->prox = NULL;
    if (lista->inicio == NULL) {          // lista vazia: nó é início e fim
        lista->inicio = nova;
        lista->fim = nova;
    } else {                             // encadeia após o último
        lista->fim->prox = nova;
        lista->fim = nova;
    }
    lista->tamanho++;
}
```

Há dois casos. Com a lista **vazia**, o novo nó é ao mesmo tempo o primeiro e o último, e **inicio** e **fim** passam a apontar para ele. Com a lista **não vazia**, o último nó atual passa a apontar para o novo (`lista->fim->prox = nova`), e só depois **fim** é atualizado. A ordem importa: se **fim** fosse atualizado antes, perderíamos a referência ao antigo último nó e ele nunca seria ligado ao novo.

Antes



Depois de `inserirFimEstoque(lista, nova)`



- ① `lista->fim->prox = nova`
- ② `lista->fim = nova`

Figura 3. Inserção no fim com ponteiro **fim** : duas atribuições, sem percorrer a lista.

A aula mostrou a inserção no **início** (`novo->prox = head`), que também é $O(1)$. Escolhemos inserir no fim porque assim a listagem do estoque aparece na ordem de cadastro, que é a ordem natural para o operador conferir as entradas.

7.3 · Remoção por código

estoque.cpp

linhas 52-73

```
// Remove pelo código. Se "copia" não for NULL, salva os dados antes do free.
bool removerBolsa(ListaEstoque* lista, const char* codigo, Bolsa* copia) {
    Bolsa* anterior = NULL;
    Bolsa* atual = lista->inicio;
    while (atual != NULL && strcmp(atual->codigo, codigo) != 0) {
        anterior = atual;
        atual = atual->prox;
    }
    if (atual == NULL) return false;

    if (anterior == NULL) lista->inicio = atual->prox; // era o primeiro
    else anterior->prox = atual->prox; // "pula" o nó removido
    if (atual == lista->fim) lista->fim = anterior; // era o último

    if (copia != NULL) {
        *copia = *atual;
        copia->prox = NULL;
    }
    free(atual);
    lista->tamanho--;
    return true;
}
```

A remoção percorre a lista com **dois ponteiros andando juntos**: `atual` e `anterior`. Isso é necessário porque, numa lista simples, um nó não sabe quem aponta para ele, e é justamente o anterior que precisa ser religado. Há três situações:

- **Nó no início** (`anterior == NULL`): o `inicio` da lista passa a ser o segundo nó.
- **Nó no meio**: `anterior->prox = atual->prox` "pula" o nó removido (Figura 4).
- **Nó no fim** (`atual == lista->fim`): além do religamento, `fim` recua para `anterior`. Se a lista tinha um único nó, `anterior` é `NULL` e a lista volta a ficar totalmente vazia (`inicio` e `fim` nulos).

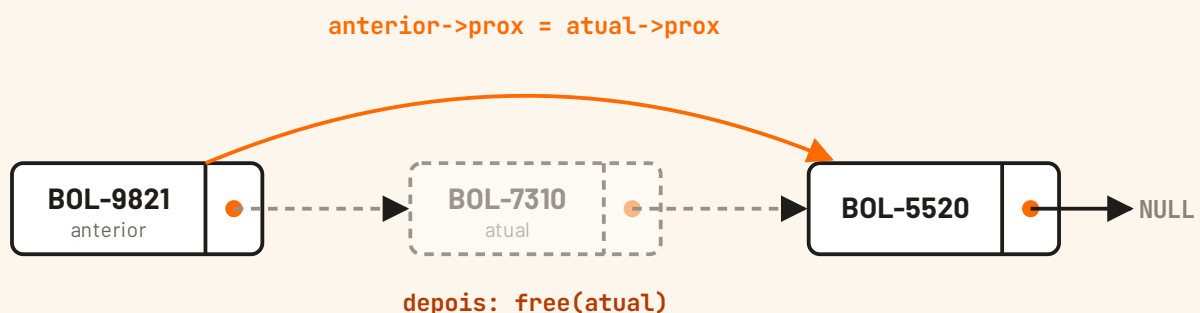


Figura 4. Remoção de um nó do meio: religa o anterior ao sucessor e só então libera o nó.

O parâmetro `copia` permite salvar os dados **antes** do `free`. O descarte passa o endereço do campo `bolsa` do nó do histórico, e assim a bolsa removida fica preservada na pilha. O desfazer de um cadastro passa `NULL`, porque não precisa guardar nada.

7.4 · Percurso, impressão e liberação

estoque.cpp

linhas 99-103

```
void imprimirBolsasComStatus(ListaEstoque* lista, const char* status) {
    for (Bolsa* atual = lista->inicio; atual != NULL; atual = atual->prox)
        if (status == NULL || strcmp(atual->status, status) == 0)
            imprimirBolsa(atual);
}
```

O percurso iterativo clássico (`for` do início até `NULL` , avançando por `prox`) serve tanto para listar todo o estoque (`status == NULL`) quanto para filtrar por status.

estoque.cpp

linhas 105-113

```
void liberarEstoque(ListaEstoque* lista) {
    Bolsa* atual = lista->inicio;
    while (atual != NULL) {
        Bolsa* proximo = atual->prox; // guarda antes de liberar
        free(atual);
        atual = proximo;
    }
    inicializarEstoque(lista);
}
```

Na liberação, o ponteiro para o próximo nó é guardado **antes** do `free` . Depois de liberar `atual` , ler `atual->prox` seria acessar memória já devolvida. Ao final, o descritor é reinicializado para não sobrar ponteiro apontando para memória liberada.

7.5 · Custo das operações da lista

Operação	Função	Custo	Observação
Inserir no fim	<code>inserirFimEstoque</code>	$O(1)$	Graças ao ponteiro <code>fim</code> ; sem ele seria $O(n)$
Buscar por código	<code>buscarBolsaRec</code>	$O(n)$	Percurso recursivo nó a nó
Remover por código	<code>removerBolsa</code>	$O(n)$	Precisa achar o nó e o anterior
Contar por status	<code>contarPorStatusRec</code>	$O(n)$	Visita todos os nós uma vez
Liberar tudo	<code>liberarEstoque</code>	$O(n)$	Um <code>free</code> por nó

08 Fila: as requisições

fila.h / fila.cpp

As requisições formam uma **fila**: a primeira que chega é a primeira atendida (FIFO). Essa é a regra de justiça do hemocentro nesta versão. Não há prioridade por validade, por tipo sanguíneo nem por urgência. A urgência é registrada e exibida, mas não reordena a fila, justamente para manter a estrutura como uma fila pura.

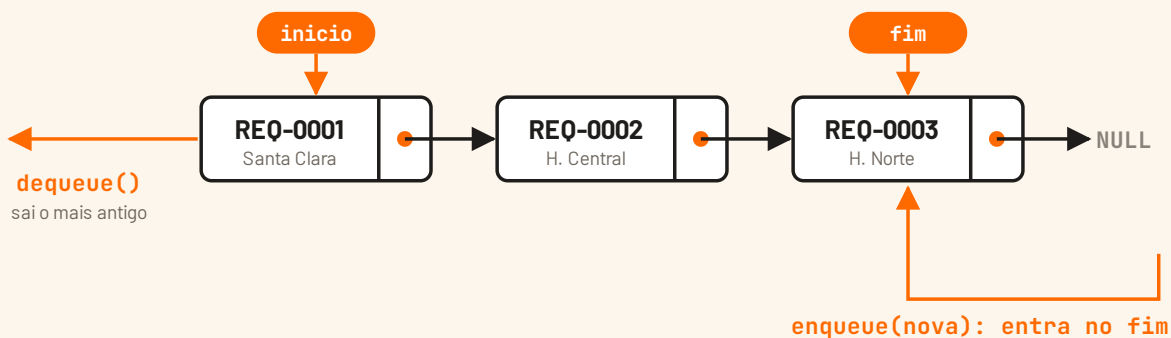


Figura 5. A fila com seus dois ponteiros: entrada pelo **fim**, saída pelo **inicio**.

8.1 · Enfileirar

fila.cpp

linhas 16-26

```
void enqueue(FilaRequisicoes* fila, Requisicao* nova) {
    nova->prox = NULL;
    if (fila->fim == NULL) {
        fila->inicio = nova;
        fila->fim = nova;
    } else {
        fila->fim->prox = nova;
        fila->fim = nova;
    }
    fila->tamanho++;
}
```

A lógica é a mesma da inserção no fim da lista (seção 7.2). A fila vazia é reconhecida por **fim == NULL**; nesse caso o novo nó vira início e fim. O **prox** do novo nó é zerado no começo, garantindo que o último elemento sempre aponte para **NULL**.

8.2 · Desenfileirar

fila.cpp

linhas 28-37

```
// Retira o primeiro da fila. Quem chama é responsável pelo free().
Requisicao* dequeue(FilaRequisicoes* fila) {
    if (fila->inicio == NULL) return NULL;
    Requisicao* removida = fila->inicio;
    fila->inicio = removida->prox;
    if (fila->inicio == NULL) fila->fim = NULL; // fila ficou vazia
    removida->prox = NULL;
    fila->tamanho--;
    return removida;
}
```

- 1 Se a fila está vazia, devolve `NULL` (não há o que atender).
- 2 Guarda o primeiro nó em `removida` e avança `inicio` para o segundo.
- 3 Se `inicio` ficou `NULL`, a fila esvaziou, então `fim` também precisa ser anulado. Sem esse passo, `fim` continuaria apontando para um nó que será liberado, e o próximo `enqueue` escreveria em memória inválida.
- 4 Devolve o nó **sem liberá-lo**: quem chamou (o atendimento) ainda precisa copiar os dados para o histórico antes do `free`.

DIFERENÇA EM RELAÇÃO AO EXEMPLO DA AULA

No slide, `enqueue` recebia e devolvia apenas o ponteiro de fim, e `dequeue` apenas o de início. Isso obriga o programa principal a manter dois ponteiros soltos em sincronia, e é fácil esquecer de atualizar um deles quando a fila esvazia. Aqui os dois ponteiros e o tamanho ficam juntos no descritor `FilaRequisicoes`, e as funções recebem o descritor por ponteiro. A lógica FIFO é idêntica; só fica mais difícil deixar a estrutura inconsistente.

8.3 · Consultar a frente

fila.cpp

linhas 39-39

```
Requisicao* peekFila(FilaRequisicoes* fila) { return fila->inicio; }
```

O atendimento primeiro **olha** a requisição da frente com `peekFila` e só a **retira** com `dequeue` depois que tudo foi validado. Se o estoque for insuficiente ou o operador cancelar, a requisição continua exatamente onde estava.

8.4 · Funções usadas apenas pelo desfazer

fila.cpp

linhas 41-48

```
// Usada só pelo "desfazer atendimento": devolve a requisição à frente da fila,  
// exatamente a posição de onde ela saiu.  
void devolverAoInicioFila(FilaRequisicoes* fila, Requisicao* req) {  
    req->prox = fila->inicio;  
    fila->inicio = req;  
    if (fila->fim == NULL) fila->fim = req;  
    fila->tamanho++;  
}
```

fila.cpp

linhas 50-65

```
// Usada só pelo "desfazer registro de requisição".  
bool removerRequisicaoPorId(FilaRequisicoes* fila, int id) {  
    Requisicao* anterior = NULL;  
    Requisicao* atual = fila->inicio;  
    while (atual != NULL && atual->id != id) {  
        anterior = atual;  
        atual = atual->prox;  
    }  
    if (atual == NULL) return false;  
    if (anterior == NULL) fila->inicio = atual->prox;  
    else anterior->prox = atual->prox;  
    if (atual == fila->fim) fila->fim = anterior;  
    free(atual);  
    fila->tamanho--;  
    return true;  
}
```

Estas duas funções não são operações normais de fila, e isso é intencional: elas existem apenas para que o desfazer restaure o estado **exato** anterior. Uma requisição que foi atendida saiu do início da fila, então desfazer o atendimento a devolve ao início. Uma requisição recém-registrada entrou no fim, então desfazer o registro a remove de lá (`removerRequisicaoPorId` , que usa a mesma técnica de dois ponteiros da seção 7.3). Como o desfazer segue a ordem inversa das operações (seção 9), a fila volta a ser idêntica ao que era, e a disciplina FIFO nunca é violada no uso normal.

09

Pilha: o histórico e o desfazer

historico.h / historico.cpp

O histórico registra toda operação que altera o sistema: cadastro de bolsa, descarte, registro de requisição e atendimento. Ele é uma **pilha** porque o uso que se faz dele, o **desfazer**, é naturalmente LIFO: a primeira coisa a ser desfeita é sempre a última que foi feita. “Mecanismos de desfazer/refazer” aparecem na aula como aplicação típica de pilhas, e aqui a aplicação é concreta: corrigir um cadastro digitado errado ou um atendimento feito com a bolsa errada.

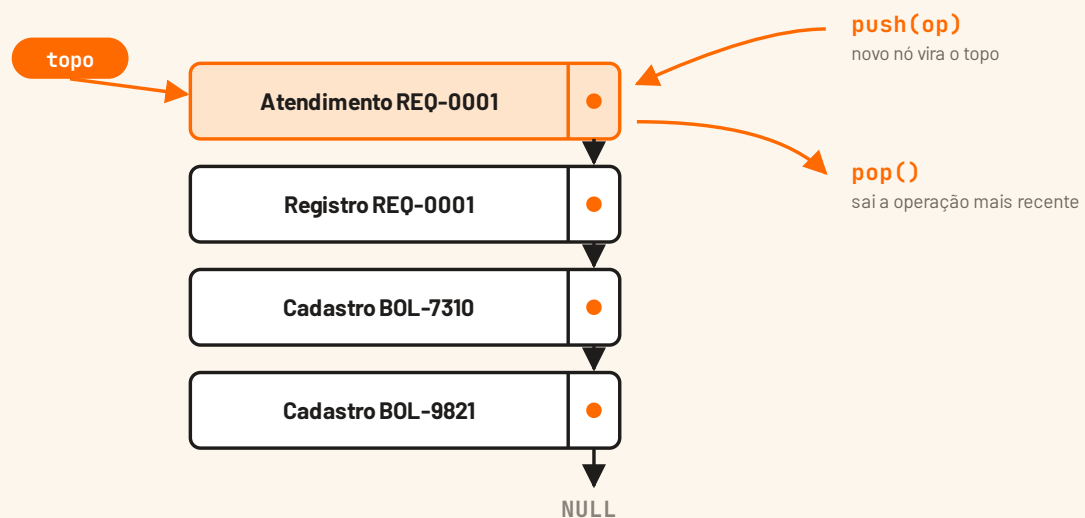


Figura 6. O histórico após a sessão de demonstração da seção 14. Só o **topo** é acessível.

9.1 · Criar, empilhar e desempilhar

historico.cpp

Linhas 15-26

```
Operacao* criarOperacao(TipoOperacao tipo, const char* descricao) {
    Operacao* op = (Operacao*)malloc(sizeof(Operacao));
    if (op == NULL) {
        cout << " Erro: memória insuficiente para registrar o histórico.\n";
        return NULL;
    }
    memset(op, 0, sizeof(Operacao));
    op->tipo = tipo;
    copiar(op->descricao, descricao, TAM_DESC);
    op->prox = NULL;
    return op;
}
```

`memset(op, 0, sizeof(Operacao))` zera o nó inteiro logo após o `malloc`. Como o nó tem campos que nem sempre são usados (uma operação de cadastro não preenche `requisicao`, por exemplo), isso garante

que nenhum campo fique com lixo de memória.

historico.cpp

linhas 28-32

```
void push(PilhaHistorico* pilha, Operacao* op) {
    op->prox = pilha->topo;    // novo nó aponta para o antigo topo
    pilha->topo = op;          // e passa a ser o topo
    pilha->tamanho++;
}
```

historico.cpp

linhas 34-41

```
Operacao* pop(PilhaHistorico* pilha) {
    if (pilha->topo == NULL) return NULL;
    Operacao* removida = pilha->topo;
    pilha->topo = removida->prox;
    removida->prox = NULL;
    pilha->tamanho--;
    return removida;          // quem chama faz o free()
}
```

`push` e `pop` são $O(1)$ e mexem apenas no topo, exatamente como os exemplos da aula. A única diferença é que `pop` **devolve** o nó em vez de liberá-lo: o desfazer ainda precisa ler a cópia guardada antes de chamar `free`.

9.2 · Desfazer a última operação

servicos.cpp

linhas 320-371

```
void desfazerUltimaOperacao(PilhaHistorico* historico, ListaEstoque* estoque,
                           FilaRequisicoes* fila) {
    cout << "\n--- Desfazer última operação ---\n";
    Operacao* op = pop(historico);
    if (op == NULL) {
        cout << " Nenhuma operação para desfazer.\n";
        return;
    }

    switch (op->tipo) {
        case OP_CADASTRO_BOLSA:
            removerBolsa(estoque, op->bolsa.codigo, NULL);
            break;

        case OP_DESCARTE_BOLSA: {
            Bolsa* b = criarBolsa(op->bolsa.codigo, op->bolsa.hemocomponente,
                                   op->bolsa.tipoSanguineo, op->bolsa.dataColeta,
                                   op->bolsa.dataValidade, op->bolsa.status);

            if (b == NULL) {
                push(historico, op);    // não perde a operação
                return;
            }
            inserirFimEstoque(estoque, b);
            break;
        }
    }
}
```

```

    case OP_NOVA_REQUISICAO:
        removerRequisicaoPorId(fila, op->requisicao.id);
        break;

    case OP_ATENDIMENTO: {
        Requisicao* r = (Requisicao*)malloc(sizeof(Requisicao));
        if (r == NULL) {
            cout << " Erro: memória insuficiente.\n";
            push(historico, op);
            return;
        }
        for (int i = 0; i < op->requisicao.qtdAlocada; i++) {
            Bolsa* b = buscarBolsaRec(estoque->inicio, op->requisicao.bolsasAlocadas[i]);
            if (b != NULL) copiar(b->status, ST_DISPONIVEL, TAM_TXT);
        }
        *r = op->requisicao;
        r->qtdAlocada = 0;
        copiar(r->status, ST_PENDENTE, TAM_TXT);
        devolverAoInicioFila(fila, r);
        break;
    }

    cout << " Desfeito: " << op->descricao << "\n";
    free(op);
}

```

Operação no topo	O que o desfazer faz
Cadastro de bolsa	Remove a bolsa da lista pelo código (<code>removerBolsa</code> com <code>copia = NULL</code>).
Descarte de bolsa	Recria o nó a partir da cópia guardada(<code>criarBolsa</code>) e o insere de volta no estoque.
Registro de requisição	Remove a requisição da fila pelo identificador.
Atendimento	Devolve as bolsas alocadas a "Disponível", recria a requisição como "Pendente de Alocação" e a coloca de volta no início da fila.

Por que o desfazer é sempre consistente

Toda alteração de estado passa pela pilha, e a pilha só permite desfazer **da mais recente para a mais antiga**. Então, quando uma operação chega ao topo, tudo o que aconteceu depois dela já foi desfeito, e o sistema está exatamente no estado em que ela o deixou. Por exemplo: uma bolsa não pode ter o cadastro desfeito enquanto está reservada, porque o atendimento que a reservou está mais acima na pilha e será desfeito antes. É a propriedade LIFO garantindo a correção do algoritmo.

Há ainda um cuidado com falhas: se a recriação de um nó falhar por falta de memória, a operação é empilhada de volta com `push`, para que não se perca e possa ser desfeita depois.

10 Recursão aplicada

Busca e contagens recursivas

A recursão, tema da aula 02, aparece em quatro funções. Todas seguem a estrutura vista em aula: um **caso base** que encerra as chamadas e uma **chamada recursiva** sobre um problema menor. Numa lista encadeada, o “problema menor” é simplesmente o restante da lista a partir de `no->prox`, o que torna listas uma estrutura naturalmente recursiva: uma lista é vazia (`NULL`) ou é um nó seguido de outra lista.

10.1 • Busca recursiva

estoque.cpp

Linhas 45-50

```
// Busca RECURSIVA: caso base = fim da lista ou código encontrado.
Bolsa* buscarBolsaRec(Bolsa* no, const char* codigo) {
    if (no == NULL) return NULL;
    if (strcmp(no->codigo, codigo) == 0) return no;
    return buscarBolsaRec(no->prox, codigo);
}
```

Há **dois casos base**: a lista acabou (`no == NULL`, a bolsa não existe) ou o nó atual é o procurado. Caso contrário, a busca continua no restante da lista. A mesma função é usada em quatro lugares: consulta por código, verificação de código duplicado no cadastro, validação das bolsas escolhidas no atendimento e restauração de status no desfazer.

10.2 • Contagens recursivas

estoque.cpp

Linhas 75-80

```
// Contagem RECURSIVA por status.
int contarPorStatusRec(Bolsa* no, const char* status) {
    if (no == NULL) return 0;
    int atual = (strcmp(no->status, status) == 0) ? 1 : 0;
    return atual + contarPorStatusRec(no->prox, status);
}
```

estoque.cpp

Linhas 82-88

```
// Contagem RECURSIVA de bolsas disponíveis de um hemocomponente.
int contarDisponiveisPorHemoRec(Bolsa* no, const char* hemocomponente) {
    if (no == NULL) return 0;
    int atual = (strcmp(no->hemocomponente, hemocomponente) == 0 &&
        strcmp(no->status, ST_DISPONIVEL) == 0) ? 1 : 0;
    return atual + contarDisponiveisPorHemoRec(no->prox, hemocomponente);
}
```

```
// Contagem RECURSIVA de operações de um tipo, do topo até a base.
int contarOperacoesRec(Operacao* no, TipoOperacao tipo) {
    if (no == NULL) return 0;
    return (no->tipo == tipo ? 1 : 0) + contarOperacoesRec(no->prox, tipo);
}
```

As três contagens têm a mesma forma: a contagem de uma lista é “1 ou 0 para o nó atual” somado à contagem do restante. A última aplica a mesma ideia à pilha, percorrendo do topo até a base. É ela que o relatório usa para contar quantos atendimentos e descartes existem no histórico.



descida: uma chamada por nó (empilhadas na pilha de execução)

retorno: os valores são somados na volta

Figura 7. Execução de `contarPorStatusRec(inicio, "Disponível")` numa lista de três bolsas.

10.3 · Recursivo × iterativo

Como na comparação feita em aula para fatorial e Fibonacci, toda função recursiva deste projeto tem um equivalente iterativo direto:

```
// Equivalente ITERATIVO de contarPorStatusRec (apenas para comparação)
int contarPorStatusIter(Bolsa* no, const char* status) {
    int total = 0;
    while (no != NULL) {
        if (strcmp(no->status, status) == 0) total++;
        no = no->prox;
    }
    return total;
}
```

Critério	Versão recursiva	Versão iterativa
Legibilidade	Espelha a definição “nó + restante da lista”	Exige variável acumuladora e controle manual do laço

Critério	Versão recursiva	Versão iterativa
Tempo	$O(n)$	$O(n)$
Memória extra	$O(n)$: uma chamada empilhada por nó	$O(1)$
Risco	Estouro da pilha de execução em listas muito grandes	Nenhum adicional

ONDE USAMOS CADA UMA

Buscas e contagens usam recursão para exercitar o conteúdo e porque o volume de uma sessão é pequeno. Já as funções de **liberação de memória** (`liberarEstoque` , `liberarFila` , `liberarHistorico`) e as de remoção foram mantidas **iterativas** de propósito: são operações críticas que precisam funcionar mesmo com estruturas grandes, sem risco de estouro da pilha de execução.

11 Casos de uso em detalhe

servicos.cpp

Os casos de uso ficam em `servicos.cpp`. Cada um segue o mesmo roteiro: **ler e validar tudo primeiro**, e só então **alterar as estruturas**. Se qualquer validação falhar, a função retorna sem ter mudado nada. Assim o sistema nunca fica “pela metade”.

11.1 · Cadastrar bolsa (US02)

- 1 **Código:** obrigatório e único. A unicidade é verificada com `buscarBolsaRec`.
- 2 **Hemocomponente:** escolhido em menu (Concentrado de Hemácias, Plaquetas, Plasma Fresco Congelado, Crioprecipitado), o que evita erro de digitação.
- 3 **Tipo ABO/Rh:** convertido para maiúsculas e conferido contra os oito formatos válidos. É validação de formato, não de compatibilidade.
- 4 **Coleta:** data real e não posterior a hoje.
- 5 **Validade:** data real, não anterior a hoje e não anterior à coleta. Caso contrário: “Data de validade inválida. Não é permitido cadastrar bolsas vencidas.”
- 6 Cria o nó da bolsa e o nó da operação. Só se os dois `malloc` derem certo a bolsa é inserida na lista e a operação é empilhada.

servicos.cpp

trecho final de cadastrarBolsa()

```
Bolsa* nova = criarBolsa(codigo, hemo, tipo, coleta, validade, ST_DISPONIVEL);
if (nova == NULL) return;

char desc[TAM_DESC];
snprintf(desc, TAM_DESC, "Cadastro da bolsa %s (%s %s)", codigo, hemo, tipo);
Operacao* op = criarOperacao(OP_CADASTRO_BOLSA, desc);
if (op == NULL) {
    free(nova);
    return;
}

inserirFimEstoque(estoque, nova);
op->bolsa = *nova;
op->bolsa.prox = NULL;
push(historico, op);
```

Repare no tratamento de falha: se a bolsa foi criada mas a operação não, a bolsa é liberada imediatamente com `free(nova)`. Sem isso, o nó ficaria alocado sem que ninguém apontasse para ele, um **vazamento de memória**.

11.2 · Registrar requisição (US01)

Segue o mesmo padrão: hospital e tipo sanguíneo são obrigatórios (se faltarem, “Requisição NÃO registrada”), a quantidade vai de 1 a 10 e a urgência é escolhida em menu. O nó é criado com `malloc` e zerado

com `memset`, recebe o identificador `proximoId` e o status "Pendente de Alocação", e entra na fila:

`servicos.cpp`

trecho de `registrarRequisicao()`

```
enqueue(fila, nova);
fila->proximoId++;
op->requisicao = *nova;
op->requisicao.prox = NULL;
push(historico, op);
```

A cópia colocada no histórico (`op->requisicao = *nova`) é feita depois do `enqueue`, e o campo `prox` da cópia é zerado: a cópia não deve carregar um ponteiro da fila, porque ela não faz parte da fila.

11.3 · Atender a próxima requisição (US03)

É o caso de uso central, porque integra fila, lista e pilha numa única operação:

- 1 `peekFila` obtém a requisição mais antiga, sem retirá-la.
- 2 `contarDisponiveisPorHemoRec` verifica se há bolsas disponíveis **do mesmo hemocomponente** em quantidade suficiente. Se não houver, a requisição fica na fila.
- 3 O sistema lista essas bolsas e o **operador escolhe** os códigos. Cada código é validado: precisa existir, estar "Disponível", ser do hemocomponente pedido e não ter sido escolhido antes no mesmo atendimento. Linha em branco cancela sem alterar nada.
- 4 Com tudo validado, as bolsas mudam para "Reservada para Despacho", a requisição é retirada com `dequeue`, copiada para o histórico e liberada com `free`.

`servicos.cpp`

verificação de estoque em `atenderProximaRequisicao()`

```
// Compara apenas o PRODUTO pedido (hemocomponente), sem regra ABO/Rh.
int disponiveis = contarDisponiveisPorHemoRec(estoque->inicio, req->hemocomponente);
if (disponiveis < req->quantidade) {
    cout << " Estoque insuficiente: há " << disponiveis << " bolsa(s) de "
         << req->hemocomponente << " disponível(is). A requisição permanece na fila.\n";
    return;
}

cout << "Bolsas disponíveis de " << req->hemocomponente << ":\n";
for (Bolsa* b = estoque->inicio; b != NULL; b = b->prox)
    if (strcmp(b->status, ST_DISPONIVEL) == 0 &&
        strcmp(b->hemocomponente, req->hemocomponente) == 0)
        imprimirBolsa(b);
```

```
// Aplica as mudanças: bolsas reservadas e requisição atendida
for (int i = 0; i < n; i++) {
    Bolsa* b = buscarBolsaRec(estoque->inicio, escolhidas[i]);
    copiar(b->status, ST_RESERVADA, TAM_TXT);
    copiar(req->bolsasAlocadas[i], escolhidas[i], TAM_COD);
}
req->qtdAlocada = n;
copiar(req->status, ST_ATENDIDA, TAM_TXT);

Requisicao* atendida = dequeue(fila); // sai do início da fila (FIFO)
op->requisicao = *atendida;           // cópia fica no histórico
op->requisicao.prox = NULL;
push(historico, op);
free(atendida);                      // libera o nó da fila
```

Onde entrariam FEFO e ABO/Rh, e por que não entram

Na versão Spring Boot, o passo 3 seria automático: o sistema ordenaria as bolsas pela validade (FEFO) e filtraria pelos tipos de doador compatíveis com o receptor (ABO/Rh). Aqui, esse passo é **manual**. A única verificação automática é se o **produto** é o mesmo que foi pedido (por exemplo, não entregar Plaquetas para um pedido de Concentrado de Hemácias). Isso é uma comparação de texto entre dois campos, e não uma regra de compatibilidade sanguínea. O tipo sanguíneo da bolsa é mostrado para o operador, mas o sistema não o usa para decidir nada.

11.4 · Descartar bolsa (US05)

```
char desc[TAM_DESC];
snprintf(desc, TAM_DESC, "Descarte da bolsa %s (motivo: %s)", codigo, motivo);
Operacao* op = criarOperacao(OP_DESCARTE_BOLSA, desc);
if (op == NULL) return;

removerBolsa(estoque, codigo, &op->bolsa); // cópia vai para o histórico, nó é liberado
push(historico, op);
```

Só bolsas “Disponíveis” podem ser descartadas: uma bolsa reservada já está comprometida com um atendimento, e descartá-la deixaria esse atendimento apontando para uma bolsa inexistente. A chamada `removerBolsa(estoque, codigo, &op->bolsa)` faz duas coisas de uma vez: grava a cópia da bolsa diretamente dentro do nó do histórico e libera o nó da lista.

11.5 · Consultas e relatório (US04 e US07)

`listarEstoque`, `consultarBolsa`, `exibirFila` e `exibirHistorico` apenas percorrem as estruturas, sem alterá-las. `exibirFila` mostra a ordem no formato usado no exercício de fila da aula (`REQ-0001 <-`

REQ-0002 <- NULL), e `exibirHistorico` marca qual operação está no topo. O relatório reúne as contagens recursivas:

`servicos.cpp`

linhas 373-386

```
// US07 - Relatório consolidado (sem previsão estatística; só contagens)
void gerarRelatorio(ListaEstoque* estoque, FilaRequisicoes* fila, PilhaHistorico* historico) {
    cout << "\n--- Relatório consolidado ---\n";
    cout << " Bolsas no estoque ..... " << estoque->tamanho << "\n";
    cout << " Disponíveis ..... " << contarPorStatusRec(estoque->inicio,
ST_DISPONIVEL) << "\n";
    cout << " Reservadas p/ despacho ..... " << contarPorStatusRec(estoque->inicio, ST_RESERVADA)
<< "\n";
    cout << " Disponíveis por hemocomponente:\n";
    for (int i = 0; i < QTD_HEMOCOMPONENTES; i++)
        cout << " " << HEMOCOMPONENTES[i] << ": "
            << contarDisponiveisPorHemoRec(estoque->inicio, HEMOCOMPONENTES[i]) << "\n";
    cout << " Requisições pendentes na fila " << fila->tamanho << "\n";
    cout << " Requisições atendidas ..... " << contarOperacoesRec(historico->topo,
OP_ATENDIMENTO) << "\n";
    cout << " Bolsas descartadas ..... " << contarOperacoesRec(historico->topo,
OP_DESCARTE_BOLSA) << "\n";
}
```

Esta é a versão, compatível com a disciplina, do painel gerencial original. Em vez de gráficos, médias e projeções estatísticas, entrega contagens exatas obtidas percorrendo as estruturas.

12 Gerenciamento de memória

Quem aloca, quem libera

Em C e C++ sem coletor de lixo, cada bloco alocado precisa ter um **dono** claro: a parte do programa responsável por liberá-lo. A tabela abaixo lista todos os pontos de alocação do projeto.

Nó alocado	Onde nasce	Quem é o dono / onde é liberado
Bolsa	<code>criarBolsa</code>	A lista. Liberado em <code>removerBolsa</code> (descarte ou desfazer de cadastro) ou em <code>liberarEstoque</code> ao sair. Se a operação correspondente não puder ser criada, o próprio <code>cadastrarBolsa</code> libera.
Requisicao	<code>registrarRequisicao</code> e desfazer de atendimento	A fila. Liberado após o <code>dequeue</code> no atendimento (depois da cópia para o histórico), em <code>removerRequisicaoPorId</code> ou em <code>liberarFila</code> .
Operacao	<code>criarOperacao</code>	A pilha. Liberado pelo desfazer após o <code>pop</code> , ou em <code>liberarHistorico</code> ao sair.

Regras seguidas em todo o código

- **Todo `malloc` é testado.** Se devolver `NULL`, a operação é abortada sem alterar as estruturas.
- **Nada é lido depois do `free`.** Os dados necessários são copiados antes (no `dequeue` do atendimento, em `removerBolsa` com cópia, no `pop` do desfazer) e o próximo ponteiro é guardado antes de liberar (nas funções de liberação).
- **Nenhum ponteiro fica apontando para memória liberada.** Ao remover o último nó, `fim` é atualizado; ao esvaziar a fila, `fim` vira `NULL`; ao liberar tudo, os descritores são reinicializados.
- **Nenhum `prox` fica sem inicializar.** Todo nó nasce com `prox = NULL` (ou é zerado com `memset`).
- **Tudo é liberado ao sair.** O `main` chama `liberarEstoque`, `liberarFila` e `liberarHistorico` antes de terminar, inclusive quando a entrada acaba de forma inesperada.

main.cpp

fim de main()

```
// Libera toda a memória alocada com malloc
liberarEstoque(&estoque);
liberarFila(&fila);
liberarHistorico(&historico);
```

Verificação com ferramentas

Além da revisão manual, o programa foi compilado com as ferramentas de diagnóstico do compilador e executado com roteiros que passam por todas as opções do menu, incluindo desfazer cada tipo de operação:

```
terminal
```

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined -o hemotrack *.cpp
```

O **AddressSanitizer** detecta acessos fora dos limites, uso de memória já liberada e liberação dupla, e o seu detector de vazamentos aponta qualquer bloco não liberado ao final. O **UndefinedBehaviorSanitizer** detecta comportamento indefinido. Nos testes realizados, nenhum dos dois reportou problemas, e a compilação com **-Wall -Wextra** não gerou avisos.

13 Complexidade das operações

Análise assintótica

A tabela resume o custo de tempo das operações, com n = número de bolsas no estoque, m = número de requisições na fila, h = número de operações no histórico e q = quantidade de bolsas de uma requisição (no máximo 10).

Estrutura	Operação	Tempo	Justificativa
Lista	Inserir no fim	$O(1)$	Acesso direto pelo ponteiro <code>fim</code>
Lista	Buscar / remover por código	$O(n)$	Pode precisar visitar todos os nós
Lista	Contagens recursivas	$O(n)$	Uma chamada por nó; memória extra $O(n)$
Fila	<code>enqueue</code> , <code>dequeue</code> , <code>peekFila</code>	$O(1)$	Acesso direto por <code>inicio</code> e <code>fim</code>
Fila	Remover por id (desfazer registro)	$O(m)$	Percurso até o nó
Pilha	<code>push</code> , <code>pop</code>	$O(1)$	Sempre no topo
Pilha	Contar operações	$O(h)$	Percurso recursivo do topo à base
Caso de uso	Cadastrar bolsa	$O(n)$	Dominado pela verificação de código duplicado
Caso de uso	Atender requisição	$O(q \cdot n)$	Cada código escolhido é buscado na lista; como $q \leq 10$, é $O(n)$ na prática
Caso de uso	Desfazer	$O(n + m)$	No pior caso, uma busca na lista ou na fila

O CUSTO DE NÃO USAR HASH

Com uma tabela hash, a busca por código seria $O(1)$ em média. Na versão C++, ela é $O(n)$, pois a lista precisa ser percorrida. Esse é exatamente o tipo de troca que a disciplina quer que o aluno perceba: a estrutura escolhida define o custo das operações. Para o volume de uma sessão de uso, $O(n)$ é perfeitamente adequado, e a seção 15 mostra como reduzir esse custo com uma estrutura da própria unidade, a ABB.

Quanto ao espaço, cada estrutura ocupa memória proporcional ao seu número de elementos: $O(n)$ para o estoque, $O(m)$ para a fila e $O(h)$ para o histórico. Nada é pré-alocado; um nó só existe enquanto seu elemento existe.

14 Execução e demonstração

OnlineGDB, g++ e roteiro de teste

14.1 · No OnlineGDB

- 1 Em **Language**, selecione **C++** (ou C++17).
- 2 Na aba `main.cpp`, apague o exemplo e cole o `main.cpp` do projeto.
- 3 Crie os demais 11 arquivos pelo botão de novo arquivo (ou envie-os pelo botão de *upload*), com os nomes **exatamente** iguais: `tipos.h`, `utils.h`, `utils.cpp`, `estoque.h`, `estoque.cpp`, `fila.h`, `fila.cpp`, `historico.h`, `historico.cpp`, `servicos.h`, `servicos.cpp`. O OnlineGDB não usa subpastas, então todos ficam no mesmo nível.
- 4 Clique em **Run**. Todos os `.cpp` são compilados e ligados juntos. Não deve existir nenhum outro arquivo com `main()`.

14.2 · Em um terminal com g++

terminal

```
g++ -std=c++17 -Wall -Wextra -o hemotrack *.cpp  
./hemotrack
```

No Windows, se os acentos aparecerem trocados, execute `chcp 65001` antes de rodar o programa para ativar UTF-8 no console.

14.3 · Roteiro de demonstração

O roteiro abaixo percorre as três estruturas e os dois cenários de erro da primeira entrega. Ele foi executado no programa real; a saída foi apenas condensada (menus omitidos), e as entradas digitadas aparecem em laranja.

```
Escolha: 1  
--- Cadastro de bolsa no estoque ---  
Código da bolsa: BOL-9821 · Hemocomponente: 2 (Plaquetas) · Tipo: A+  
Coleta: 01/09/2026 · Validade: 31/12/2030  
Bolsa BOL-9821 adicionada ao estoque com status "Disponível".  
Saldo disponível de Plaquetas: 1 bolsa(s).  
(BOL-7310, Plaquetas, 0-, validade 20/01/2031 cadastrada da mesma forma)  
  
Escolha: 1 BOL-0042 ... Validade: 01/01/2020  
Data de validade inválida. Não é permitido cadastrar bolsas vencidas.  
  
Escolha: 5 Hospital Santa Clara · Plaquetas · 0+ · 2 bolsas · Emergência  
Requisição cadastrada com status "Pendente de Alocação".  
Identificador de rastreamento: REQ-0001  
Posição na fila: 1  
  
Escolha: 5 Hospital Boa Vista · Hemácias · tipo sanguíneo: (em branco)
```

Requisição NÃO registrada: o tipo sanguíneo do paciente é obrigatório.

Escolha: 7

--- Atendimento da próxima requisição ---

REQ-0001 | Hospital Santa Clara | Plaquetas | 0+ | 2 bolsa(s) | Emergência | Pendente de Alocação

Bolsas disponíveis de Plaquetas:

[BOL-9821] Plaquetas | A+ | coleta 01/09/2026 | validade 31/12/2030 | Disponível

[BOL-7310] Plaquetas | 0- | coleta 05/09/2026 | validade 20/01/2031 | Disponível

Bolsa 1 de 2: **BOL-9821** Bolsa 2 de 2: **BOL-7310**

REQ-0001 atendida. Bolsas com status "Reservada para Despacho": BOL-9821, BOL-7310

Escolha: 8

1. Atendimento da requisição REQ-0001 (2 bolsa(s)) <- topo

2. Registro da requisição REQ-0001 (Hospital Santa Clara)

3. Cadastro da bolsa BOL-7310 (Plaquetas 0-)

4. Cadastro da bolsa BOL-9821 (Plaquetas A+)

Escolha: 9

Desfeito: Atendimento da requisição REQ-0001 (2 bolsa(s))

Escolha: 0

Encerrando...

Memória liberada. Até logo!

Passo do roteiro	O que demonstra
Cadastros BOL-9821 e BOL-7310	Inserção no fim da lista e contagem recursiva do saldo
Cadastro BOL-0042 vencida	Validação antes de qualquer alocação (US02, cenário 2)
Requisição REQ-0001	enqueue e geração do identificador (US01, cenário 1)
Requisição sem tipo sanguíneo	Campo obrigatório: nada é alocado nem enfileirado (US01, cenário 2)
Atendimento	peekFila , filtro por hemocomponente, escolha manual, dequeue e free
Histórico	Pilha com quatro operações e o topo indicado
Desfazer	pop : bolsas voltam a "Disponível" e REQ-0001 volta ao início da fila
Sair	Liberação de toda a memória

15 Conclusão e evoluções

Síntese e próximos passos

A versão C++ do HemoTrack cumpre o que a Unidade 1 de Algoritmos e Estrutura de Dados pede: o estoque é uma lista encadeada, as requisições formam uma fila FIFO e o histórico é uma pilha que dá sentido prático ao LIFO por meio do desfazer. Todas as estruturas foram construídas nó a nó, com `malloc`, `free` e ponteiros explícitos, e a recursão aparece onde ela expressa melhor o algoritmo.

A principal decisão de projeto foi **separar o que é da disciplina do que é do produto**. FEFO, compatibilidade ABO/Rh, hash, roteirização e análise preditiva continuam fazendo parte do HemoTrack, na versão Java com Spring Boot desenvolvida em Programação Orientada a Objetos, onde esses recursos eram permitidos. Aqui, onde eram proibidos, o mesmo domínio foi reorganizado para que cada funcionalidade existisse por causa de uma estrutura de dados, e não apesar dela.

Evoluções possíveis dentro do conteúdo da disciplina

Árvores binárias e árvores binárias de busca também fazem parte do conteúdo da unidade, mas não estavam entre as estruturas exigidas nesta entrega (lista, fila e pilha), por isso não foram implementadas. A primeira linha da tabela mostra onde elas se encaixariam.

Evolução	Benefício
Índice do estoque em ABB	Uma árvore binária de busca ordenada pelo código tornaria a busca $O(\log n)$ no caso médio, e o percurso em-ordem listaria as bolsas em ordem alfabética de código. É o conteúdo das aulas 04 e 05 aplicado ao sistema.
Lista duplamente encadeada	Com ponteiro <code>ant</code> , a remoção não precisaria de dois ponteiros andando juntos e a lista poderia ser percorrida nos dois sentidos.
Refazer (<i>redo</i>)	Uma segunda pilha receberia as operações desfeitas; uma nova operação a esvaziaria. É o mecanismo desfazer/refazer citado na aula de pilhas.
Persistência em arquivo	Gravar as estruturas em arquivo ao sair e reconstruí-las ao iniciar, percorrendo cada estrutura uma vez.

Em uma frase

Na versão Spring Boot, o HemoTrack mostra **o que** o sistema faz. Na versão C++, mostra **como os dados vivem na memória** enquanto ele faz, e é isso que esta disciplina avalia.